

Machine-in-the-Tunnel: Practical Connection Inference, DNS Injection, and Defenses for Modern VPNs

Beau Kujath
Arizona State University
Breakpointing Bad
Albuquerque, New Mexico, USA
beau@breakpointingbad.com

William J. Tolley
Hampden-Sydney College
Breakpointing Bad
Farmville, Virginia, USA
william@breakpointingbad.com

Jedidiah R. Crandall
Arizona State University
Breakpointing Bad
Tempe, Arizona, USA
jed@breakpointingbad.com

Abstract

According to 2025 statistics, approximately 33% of Internet users have used a VPN, and this number continues to grow. When a user connects to a typical VPN server, all of their traffic is *tunneled* through the server, which forwards the traffic on the user's behalf. This encrypted tunnel protects VPN users from a wide range of common real-world adversaries, including ISPs, governments, local network attackers, and upstream eavesdroppers. However, little research has examined what an *active* in-path adversary can infer about traffic traversing such a tunnel. In this paper, we demonstrate that a blind in-path attacker (*e.g.*, a compromised router between the VPN client and VPN server) can reliably *confirm* whether a VPN user is communicating through the encrypted tunnel with an IP address in a set large enough to cover 50 major social-media and news domains.

Since modern content-distribution infrastructure causes many services to share IP addresses, identifying communication with an IP address alone does not reveal the specific service being accessed. We therefore extend the attack by poisoning the DNS entry for the target domain, enabling an in-path attacker to observe the victim's TLS Server Name Indication (SNI). Depending on the operating system, the attacker can infer requests for specific domains associated with the target IP address, poison the DNS cache, or induce a denial-of-service condition. Finally, we design and implement lightweight helper modules for Linux VPN servers that leverage existing connection-tracking mechanisms to detect and block spoofed packets injected by in-path adversaries, mitigating these attacks across multiple transport- and application-layer protocols, including TCP, DNS, QUIC, and DTLS.

CCS Concepts

• **Security and privacy**; • **Networks** → **Network privacy and anonymity**;

Keywords

VPN security, privacy, connection inference, DNS cache poisoning, side-channel attacks

ACM Reference Format:

Beau Kujath, William J. Tolley, and Jedidiah R. Crandall. 2026. Machine-in-the-Tunnel: Practical Connection Inference, DNS Injection, and Defenses for Modern VPNs. In *25th Workshop on Privacy in the Electronic Society (WPES '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3847192.3847368>

1 Introduction

VPNs encrypt each outgoing IP packet from a user's device and tunnel it to a VPN server, which then forwards it to its final destination. To upstream observers, the user appears to be communicating only with the VPN server's IP, hiding the true destinations. This obfuscation protects users from adversaries such as ISPs, governments or local attackers. If an adversary could reliably confirm the user's activity through the tunnel, it would undermine the VPN's core purpose of hiding the IP addresses, domains, and services a user is accessing.

VPNs are increasingly difficult to access in countries with strict information controls such as China, Iran, and Russia, where upstream routers enforce domain blocklists [3, 9, 11, 14, 27, 46]. These states maintain large blocklists of VPN and Tor IPs. For example, China's Great Firewall uses centralized filtering of IPs and domains [9, 11], while Russia's TSPU system operates across disparate ISPs to drop or reset circumvention-related traffic [27, 46]. In more permissive regions, VPNs are commonly used to bypass geofencing or evade ISP surveillance. Regardless of whether VPN usage is motivated by privacy, censorship circumvention, or geofencing, concealing the destinations and services being accessed remains a critical security concern.

Recent legal developments have introduced a more subtle threat model than simply blocking VPNs outright. In 2024, Brazilian authorities imposed fines of approximately \$8,900 USD per day on users accessing the formerly blocked web app at *x.com* through VPNs [28]. Similarly, Russia has passed legislation allowing prosecution or penalties for accessing banned services through circumvention tools [44]. These policies create a strong incentive for upstream adversaries to confirm whether a VPN user is actively communicating with a specific service, even when the traffic itself remains encrypted.

This threat model is particularly relevant for mobile users. More than 60% of Brazilians access the Internet exclusively through smartphones [4], and Linux-based Android devices accounted for an estimated 81.86% of the country's mobile market share as of January 2026 [38]. In practice, a VPN user connected to a commercial provider such as NordVPN would expect all traffic leaving the device to appear as a single encrypted connection to the VPN server,



This work is licensed under a Creative Commons Attribution 4.0 International License. *WPES '26, The Hague, Netherlands*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-3026-9/2026/11
<https://doi.org/10.1145/3847192.3847368>

preventing observers from confirming access to a particular website. Modern VPNs also claim protection against common MITM attacks, including DNS injection and connection inference.

Despite these protections, researchers have repeatedly demonstrated methods for breaking the isolation guarantees of encrypted VPN tunnels [16, 30, 39, 41, 48]. In this work, we investigate how such vulnerabilities could be used in practice by an active in-path adversary seeking to confirm access to specific websites through a VPN tunnel. We focus on a side-channel-based connection inference vulnerability described by Tolley *et al.* [39], which remains unpatched in many Linux environments and is tracked as CVE-2019-14899 [40]. Using this vulnerability, we develop a framework to evaluate the practical privacy implications of VPN tunnel attacks in environments where users face legal or financial penalties for accessing specific services.

The attack first performs connection inference to identify communication with a target server IP and subsequently leverages DNS injection to associate that communication with a specific domain when necessary. Finally, we develop a set of conntrack helper modules that can be deployed on a Linux VPN server to defend against the attacks presented in this paper. We show how the VPN server can leverage existing connection-tracking mechanisms, such as conntrack, to identify and block spoofed packets injected by an in-path middlebox before they are forwarded to the VPN client. Our approach is effective across a range of transport and application protocols, including TCP, DNS, QUIC, and DTLS.

In order to demonstrate the practicality of the attack, we conduct experiments and information-theoretic analysis that lead us to the following outcomes:

- An active attacker can reliably infer and *confirm* a connection through the VPN tunnel to a short list of IP addresses.
- The attacker can drop all DNS replies for a target domain to create an injection window of at least 6 seconds.
- A dedicated attacker can reliably poison the DNS cache of a Linux-based VPN client using a confirmed target domain and forcing traffic for that domain to bypass the VPN tunnel, exposing the SNI in cleartext.
- Android is also vulnerable to DNS cache poisoning.
- The attacker can infer domains requested within the encrypted tunnel on macOS and create a denial-of-service condition on Windows.
- The attack is robust to packet-size obfuscation (e.g., padding) because its timing channel alone still leaks port information, so only protocol-aware defenses that shorten the attack window are effective in stopping it.
- We design and implement Linux VPN server helper modules that leverage conntrack to block spoofed packets injected by in-path middleboxes, providing an effective defense across TCP, DNS, QUIC, and DTLS connection flows.

2 Threat Model

In this section, we detail the requirements and goals of the malicious middlebox that is targeting a single encrypted connection between a VPN user and their VPN server.

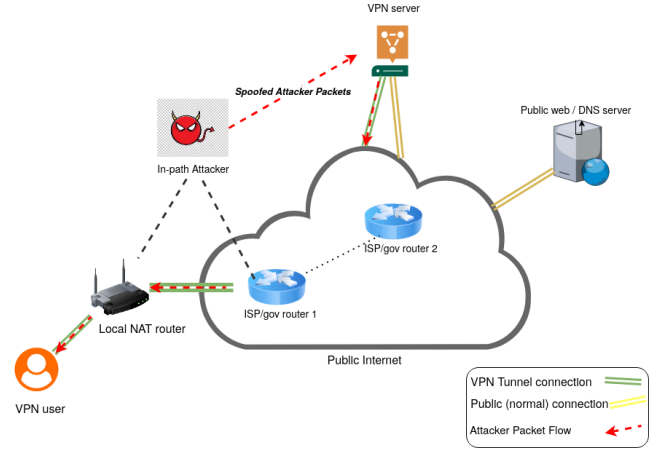


Figure 1: Threat model for blind in-path attacks targeting a VPN client

2.1 Attacker Requirements

As shown in Figure 1, the threat model for the attacks presented includes an “in-path” active attacker or middlebox that is targeting a VPN user. An in-path attacker sits somewhere along the network path between the VPN server and VPN client. The attacker is responsible for forwarding the encrypted traffic between the two endpoints at some hop in the network path.

This attacker could be part of the ISP that is routing traffic on behalf of the user or even a state-controlled backbone infrastructure router like many of the “chokepoints” found in centralized Internet architectures including those used in China or Iran [11]. Concretely, inline/forwarding-capable deployments that match our in-path model include Russia’s TSPU [46], China’s “Great Cannon” [26], and Sandvine/Procera PacketLogic deployments documented in Turkey and Egypt [25]. By contrast, on-path (or “machine-on-the-side”) systems that can observe and inject but do not forward—e.g., the Great Firewall’s RST injection [11] and website-targeted false-content injection/QUANTUMINSERT-style races [17, 31], do not satisfy our requirement. The attacker could instead sit closer to the VPN user or control an ISP node a few hops upstream, similar to documented TSPU deployments in Russia [46]. The attacker could also control a malicious NAT device to which the user connects at a public coffee shop. The in-path attacker could also sit on the same local network as the victim and perform standard ARP-poisoning techniques to become in-path [5, 22].

The first requirement for the attacker is the ability to spoof IP packets which is a standard capability considered in network-attack research and available to sophisticated adversaries. Next, the attacker must be “in-path” and responsible for forwarding all the victim’s traffic at some hop in the network. This implies the attacker can choose to drop or forward any of the encrypted packets it is observing between the VPN client and VPN server in real time. The attacker is considered “blind” since the VPN connection is encrypted and it cannot see the packet details besides packet size and timing. Lastly, the attack requires a NAT-based VPN server that uses conntrack or similar connection-tracking systems to map

internal connection flows to external flows, such as OpenVPN or WireGuard.

2.2 Attacker Goals

1. *Confirm an active connection:* First, the attacker seeks to determine whether the target VPN user has an active connection to any server IP address associated with a monitored set of domains. For example, the attacker attempts to confirm whether the VPN user has an active TCP connection to one of the server IP addresses associated with `tiktok.com`.

In a real-world scenario, the scan list of IP addresses used by the adversary would vary depending on the region or network the user is located in and which websites the adversary is interested in monitoring. For example, in the threat model associated with Brazilian authorities targeting `x.com` VPN users, the adversary’s scan list might only include the primary IP addresses being used by `x.com`. For our experiments in section 5.2, we sample from the top 50 social-media and news domains to determine how consistently they connect to the same public IP(s) across browser sessions. We show that many, including `x.com`, reliably use a small number of IP addresses, allowing the adversary to scan the candidate set quickly.

2. *Inject DNS entry into VPN client’s operating-system cache:* Once the attacker has confirmed an active connection to an IP address associated with a target domain, the attacker then attempts to escalate the attack by injecting an arbitrary DNS response for a domain D into the victim’s DNS cache. The attacker will attempt to use information obtained from the first attack step to actively drop the legitimate tunneled DNS responses for D . The attacker’s blind drop rules will, in turn, create a DNS “lookup loop” where the VPN client is forced to repeatedly query the unresolved domain. Because the UDP source port remains open while the client awaits a valid response, the attacker has time to identify the port used by the outstanding DNS query.

The final goal of the DNS injection is to map a target domain D to the IP address of the VPN server in use. Because common full-tunnel VPN configurations maintain a routing rule in place that sends any packet to the VPN server IP out the default wireless or Ethernet interface, this cached mapping will cause traffic destined for domain D to bypass the tunnel altogether [39, 48]. Now the attacker can *confirm* access to D by observing the cleartext SNI for domain D sent by the VPN client in the TLS ClientHello.

Most modern browsers, including Chrome and Firefox, enforce maximum DNS TTLs of 60 or 120 seconds for cached replies. As a result, every minute or two the browser will request a DNS mapping from the operating system for a domain that is being used. We show in Section 5.4 that many modern operating systems, including those using `systemd-resolved` cache DNS replies for up to two hours by default. Therefore, once the attacker confirms an active connection to a website, they can expect a future DNS query to be sent for that domain within the next two minutes. This subsequent query creates an opportunity for the attacker to attempt a successful DNS injection to the OS cache where they can cause the malicious mapping to persist for up to the resolver’s maximum accepted TTL. Furthermore, we show that most modern operating systems will query the same background service domains at regular intervals providing more injection opportunities for the attacker.

3 Background

As of 2025, 33% of Internet users worldwide have used a VPN [42]. According to a survey by CNET, 43% of American adults use VPNs and 47% report that their primary reason is to protect their privacy and online activity [29]. According to real-time statistics by VPN Gate, VPN usage is even more prevalent in other countries [12]. According to VPN Gate’s public country-level connection and traffic statistics, the United States ranks fifth. Countries with strict information controls, such as Russia and Iran, lead the rankings where users may need VPNs to access specific applications and websites. Users across the globe rely on VPNs for enhanced security, privacy, and access to websites and resources that would otherwise be unavailable. This section reviews the VPN and DNS mechanisms relevant to our attacks.

3.1 Connection Tracking by VPN Servers

In a typical Internet connection, only the client and the web server maintain connection state. When a user connects through a VPN, the VPN server must also maintain connection state to map flows between the “internal” VPN client and the “external” public Internet. Without this state, the VPN server would be unable to determine which VPN client should receive packets arriving from external servers.

`conntrack` is a core component of the Linux kernel’s networking stack provided by the `nf_conntrack` module. It forms the basis for features such as NAT and stateful packet filtering [35]. Remote-access VPN servers use `conntrack` to maintain NAT state, mapping “internal” VPN client flows to “external” Internet flows.

For each incoming packet, the VPN server attempts to match it to an entry in the `conntrack` table. If a matching flow exists, the packet is forwarded to the corresponding VPN client. Otherwise, it is discarded. Packets are encrypted or decrypted as they enter or leave the “internal” VPN tunnel and are forwarded between the VPN client and the “external” Internet.

3.2 Connection Inference and Confirmation

Connection inference is the process by which an adversary determines whether a target IP address has an active network connection to another IP address. A connection could be a long-lived TCP connection to a website like `x.com` or a short-lived UDP flow used by a stateless protocol such as DNS. While a connection is active for a VPN user, the TCP or UDP connection state information will be stored by `conntrack`, or the operating system’s connection-tracking framework, on the VPN server.

When a user is not connected to a VPN, a middlebox can directly observe the user’s connections since there is no encryption or “tunneling” to one IP. They are able to see the exact IP addresses with which the user is communicating, the ports being used, and the network protocol. Even when HTTPS is used, the middlebox can still view the exact domain being accessed by the user by viewing the Server Name Indication (SNI) in the TLS ClientHello message [8].

However, when the user is using a privacy tool such as a VPN, all their traffic should look like a single encrypted connection to one IP address; therefore, these tunneled connections must be *inferred* through side channels. This forces the adversary to exploit side

channels exposed by the VPN client or the VPN server's operating system to probe for these active connections.

We define connection **confirmation** as determining, with a false-positive rate below 0.01%, that the inferred connection exists. In the following attacks, this can be done by sending repeated follow-up “recheck” probes. The more packets that are sent to confirm and verify the connection, the lower the probability of a false positive. For example, if the attacker infers a connection and sends 300 recheck packets to the connection 5-tuple (*src*, *sport*, *dst*, *dport*, *protocol*) then the attacker should observe 300 corresponding packets forwarded through the tunnel to the VPN client. If the adversary can reliably confirm that a user is connected to a specific IP through the VPN server, this confirmation could lead to both social and legal consequences depending on the region or network.

3.3 DNS Injection and Operating-System Handling

The DNS protocol allows a user to receive a list of IP addresses that can be used to connect to a requested domain. DNS injection is one of the simplest, yet harmful network attacks that can still be performed by a middlebox in most standard network environments today when the user is not connected to a VPN. Standard UDP-based DNS traffic is sent in cleartext (unencrypted), unlike encrypted application protocols like HTTPS. Therefore, if an attacker is in-path between a user and their public DNS server, the attacker can inject a malicious DNS response after observing the victim's outgoing DNS request. The attacker can simply drop the legitimate DNS response from the DNS server to ensure the victim only receives their malicious response.

DNS injection is less difficult than TCP injection since the attacker only needs to know the source port and transaction ID in the DNS request for the injected response to be accepted. TCP has additional packet fields such as the sequence and acknowledgment numbers with much larger value ranges (32 bits) than the DNS transaction ID, which is only 16 bits and therefore has 65,536 possible values. However, when the victim is connected to a VPN, the original DNS query and the real response from the DNS server will be encrypted within the tunnel. Sections 4 and 5 show that even with a valid VPN connection, a sophisticated attacker can still identify the outgoing encrypted query using observable tunnel metadata, drop the legitimate replies, and then attempt to inject a malicious DNS response for that domain.

3.3.1 Operating-System DNS Handling. The default DNS resolver on each operating system determines the maximum number of DNS retransmissions and the timeout period for a failed DNS query. For example, the default resolver on Windows 10/11 waits two seconds per query and allows three retransmissions. Therefore, when an in-path attacker drops the valid DNS replies, the source port remains available for approximately eight seconds. This resolver behavior plays an important role in blind DNS injection, as the attacker must infer the source port and brute-force the 65,536 possible DNS transaction IDs within this window.

Each operating system also has different DNS caching behavior, determined by its default DNS resolver. For example, on Ubuntu 24.04 the default system cache implemented by `systemd-resolved` caps cached DNS TTLs at two hours. If a response specifies a larger

TTL, the resolver caps it at 7,200 seconds ($60 \times 60 \times 2$). On the other hand, macOS will keep a domain in its cache for a maximum of seven days. This behavior determines how long a successful DNS injection can remain in the cache and affect the VPN client.

3.4 Passive versus Active Attacks

Previous studies have examined passive techniques to fingerprint websites through a VPN or encrypted tunnel [15, 32, 36, 37]. Passive attacks imply the attacker does not need to actively interfere or spoof any packets to the target, but can passively collect and monitor traffic to make inferences about connections. These works generally rely on deep-learning techniques to train classifiers that can identify visits to a small set of “monitored” websites. However, they may exhibit high false-positive rates, require large datasets and substantial model-training time, and perform poorly in “open-world” scenarios involving dynamic users and thousands of unmonitored websites within the encrypted traffic stream [19].

Active attacks involve dropping or spoofing packets to interfere with the target. They require active interference and additional network traffic, but allow the attacker to verify and confirm information in real time. Tolley et al. [39] showed that active attacks can be used to target a VPN user to infer connections and the exact TCP sequence and acknowledgment numbers in use through probing. In this paper, we extend the active server-side attack by evaluating its practicality when selectively dropping packets, testing it against a large set of real-world websites, examining whether it remains effective against padded VPN protocols such as WireGuard, and determining whether it can be escalated to DNS injection on modern systems.

4 Attack Design

The attack model assumes that a VPN client has established a connection to a VPN server. An in-path middlebox attacker, referred to as Mallory, resides at an intermediate hop between the client and server and relays all encrypted traffic exchanged between them. Mallory's attack consists of three main phases:

- (1) Prerequisites: Mallory identifies the VPN's packet-length encapsulation scheme and derives the corresponding packet-length model $E(p)$.
- (2) Active connection scan: Mallory attempts to confirm an active connection 5-tuple associated with a particular server IP address.
- (3) DNS injection attempt: Mallory attempts to inject a malicious DNS reply to the VPN client's operating-system cache.

4.1 Attack Prerequisites

The attack targets remote-access, full-tunnel VPNs. Before launching the active attack, Mallory performs a passive reconnaissance phase that can be conducted over an extended period without interacting with the victim. During this phase, the attacker identifies the VPN protocol in use, including whether it employs static encapsulation or bucket-based packet padding through passive analysis of encrypted packet-length distributions. In particular, Mallory distinguishes VPNs using OpenVPN's default static encapsulation from those using the default 16-byte bucket padding used by WireGuard.

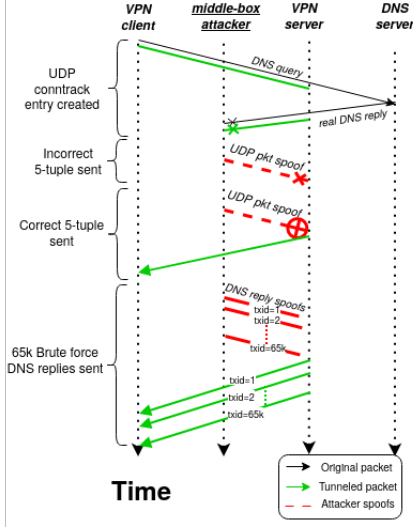


Figure 2: Attack diagram for blind in-path DNS injection

Appendix B.1.2 describes the VPN identification techniques and passive traffic analysis used during this reconnaissance phase.

After determining the encapsulation scheme, Mallory derives the corresponding packet-length model, denoted $E(p)$, which predicts the encrypted tunnel length of an original IP packet of length p . This model enables Mallory to precompute the encrypted sizes of arbitrary TCP and UDP packets, including spoofed probes, DNS queries, and DNS replies required for the connection-inference and DNS-injection stages described below. Appendix B.1.1 provides the packet-length equations for both static and bucket-padded VPN protocols.

4.2 Form Scan List

Mallory first chooses the websites or IP addresses for which it wants to test whether the VPN user has an active connection. This choice will be heavily influenced by the user’s region and which websites are considered “sensitive” or otherwise of interest to Mallory. In order to get an idea of how serious this threat can be in practice, our custom scan list includes popular websites from countries around the world.

We selected a custom test list of 50 domains composed of the most popular social-media websites (by monthly active users) and news websites globally. The list includes 30 social media and 20 news websites. To assess this threat model based on the daily fine in Brazil, we manually chose a 50-website list to include a wide variety of platforms and measured how consistently they reuse public IP addresses and how long modern browsers keep the resulting connections open in the background. Mallory could also choose to include domains likely to be queried by background operating-system services on the target device such as `connectivity-check.ubuntu.com`, if the goal is merely to confirm any connection that can provide an opportunity for DNS injection.

Because social-media applications and news websites allow users to disseminate information at scale, these websites are often blocked in countries with information controls. Therefore, it is plausible for

an adversary to form a list of the most popular social websites that could be used to communicate or disseminate information beyond national borders.

After selecting the 50 domains, the next step for Mallory is to form the IP scan list S , which will be used later to confirm connections to one of the sites. Mallory wants to optimize the list S to include only the IP addresses with the highest observed probability of being used by each website. In Section 5.2 below, we measure how consistently each of the 50 websites uses the same IPv4 addresses when accessed in a browser. Using these measurements, Mallory can construct a small scan list S containing IP addresses likely to be used for connections to the selected domains.

4.3 Active Connection Inference and Confirmation

Using the IP scan list S , Mallory iterates over each IP, s , to probe the VPN server to which the target VPN user is connected. Because most modern web services use port 443 as the destination port for HTTPS traffic, Mallory spoofs each probe as originating from source IP s and source port 443. The destination IP is the VPN server’s public IP address, and the destination port is varied across the range of ephemeral ports that the VPN server may have assigned to the connection. On Linux, the default ephemeral port range is typically 32768–60999, while Windows typically uses 49152–65535. macOS uses a dynamically allocated ephemeral port range rather than a single fixed range. The relevant range depends on the operating system and NAT configuration of the VPN server.

Mallory spoofs the same TCP packet from $s:443$ to $VpnServerIP:e$ while iterating over each possible ephemeral port (e) that could be used. For TCP inference, the spoofed probes must be SYN-ACK packets. SYN-ACK packets are the only TCP packet type that will bypass valid sequence and acknowledgment value checks in the spoofed packets. Mallory can choose any valid IP packet length for the probes and use the E function to calculate the exact size that should be observed if the packet matches an existing connection in the VPN server’s conntrack table and is forwarded back to the VPN client.

Mallory exploits the fact that, regardless of the content of a spoofed packet, the VPN server will only match and forward packets that correspond to an active connection in the conntrack table. Most spoofed packets are dropped by the VPN server; only those matching an existing conntrack entry are forwarded. Matched packets are forwarded through the encrypted tunnel to the VPN client, where Mallory can observe them. Since Mallory only observes the encrypted tunnel between the target VPN client’s public IP address and the VPN server’s public IP address, it does not matter whether multiple VPN users are connected to the same web server IP through the VPN server.

Confirmation and Fine-Tuning. To reduce false positives caused by legitimate tunnel traffic observed by Mallory’s monitoring script when valid packets between the VPN client and VPN server are exchanged, the attacker can carefully choose a *needle* size for the probes. For example, the attacker can choose to send only 800-byte packets so that observing an encrypted packet of size $E(800)$ identifies a candidate port for rescanning. In practice, the attack script can use a range of needle packet sizes in each round,

which allows it to increase the speed of the inference by gathering more information in each probing round, while only slightly increasing the rate of false positives.

Once a potential active connection 5-tuple is found, Mallory can repeatedly *confirm* that the match is correct and still exists in the *conntrack* table by sending additional probes to the same 5-tuple. For example, if Mallory sends k 800-byte packets to the 5-tuple, then Mallory should expect to observe k corresponding encrypted packets of size $E(800)$. This establishes that the connection remains active in the *conntrack* table and that the VPN user is still connected to server IP address s .

4.4 DNS Injection Attempt

At this point, we can assume Mallory has confirmed an active connection to an IP address associated with a domain D or has selected a periodic background DNS query for a domain D made by the VPN target's operating system. We detail below in Section 5.4.1 and Table 2 the most consistently queried periodic domain by each of several common operating systems we tested, including Ubuntu 24.04, Fedora 42, Debian 13, Windows 10 and 11, and macOS Sequoia. However, only Linux-based systems accept replies containing all possible transaction IDs that are brute-forced by Mallory in this step. Therefore, Windows 10 and 11 and macOS Sequoia are not vulnerable to the complete DNS-injection attack.

Now that Mallory has selected the domain D that it will target for the injection, Mallory precomputes the encrypted packet sizes of both the DNS queries and replies for D using the formula E . We developed a simple Python script that captures x DNS lookups and responses for the domain and generates the *iptables* rules needed to drop the encrypted reply packets [24]. The VPN client repeatedly retransmits the DNS query for D while waiting for a valid reply. Depending on the operating system resolver's maximum number of retransmissions and timeout period for each lookup, the UDP source port remains open while awaiting a DNS reply for 6–120 seconds. The VPN client's operating system repeatedly sends DNS queries for the domain D since it is not getting a reply. We refer to this repeated querying behavior as a *lookup loop*.

Even when the client's system falls back to its backup DNS servers, Mallory's *iptables* drop rule ensures that every legitimate DNS reply is dropped, giving Mallory sufficient time to perform a malicious injection attempt. After placing the drop rules, Mallory will use the observed packet sizes of both $E(dnsQuery)$ and $E(dnsReply)$ to trigger the injection attempt.

The DNS injection script begins by rapidly sending spoofed UDP packets to the VPN server that appear to originate from the DNS server using the IP address of the VPN client's DNS server as the source address—for example, 8.8.8.8 for Google DNS—and source port 53. Mallory uses the same steps described in Section 4.3 for TCP confirmation to infer and confirm the UDP flow that corresponds to the target DNS lookup. The scan list S consists only of candidate DNS servers being used by the client or VPN server. Once the primary DNS server is confirmed, the scan list will only need to include this single IP address. The scan identifies any open UDP source ports used by the client to communicate with a DNS server on destination port 53.

Mallory can now craft the full malicious DNS reply for D to send to the VPN server on the confirmed port - c_{port} - that will be forwarded back to the VPN client. Mallory will continually spoof from $DNSServerIP : 53$ to $VPNServerIP : c_{port}$ with a crafted malicious DNS reply. The reply will include the domain D with an arbitrary malicious DNS mapping, such as a CNAME record mapping D to attacker.example, accompanied by an address record mapping that name to $VPNServerIP$. Mallory does not know the DNS transaction ID used in the query, so Mallory brute-forces it by sending all 65,536 possible transaction IDs in 65,536 packets. This final brute force step can be performed reliably in our experimental threat environment in under 2.5 seconds.

In the tested Linux-based systems, the VPN client does not close the UDP source port after receiving large numbers of replies with incorrect transaction IDs. Once Mallory successfully injects and caches a mapping from D to $VPNServerIP$ through the cached DNS injection, Mallory then installs a DNAT rule to redirect all traffic destined for the injected IP address to a legitimate server IP address for D . Thus, when the domain is accessed over HTTPS, the TLS handshake will succeed and Mallory can observe the SNI in the ClientHello messages sent outside the tunnel. Because the injection causes only traffic intended for D to bypass the tunnel, Mallory needs to forward only this out-of-tunnel flow to a legitimate server IP address for D . Mallory can then observe the flow as though the target were not using a VPN.

5 Measurements and Experimental Validation

To test the viability of this attack, we set up two VPN servers hosted on a cloud platform. We configured one Ubuntu 22.04 VM as a default OpenVPN server by following a standard setup tutorial [7]. We configured the second cloud Ubuntu 22.04 VM to be a standard WireGuard VPN server [6]. The WireGuard server represents a VPN protocol with *bucket* padding, while the OpenVPN server represents a VPN protocol with *static* padding.

5.1 Setup

For the VPN clients, we used our research network to create multiple VMs that served as victim VPN clients. The attack can apply to all operating systems we examined since each will exchange consistent packet sizes depending on the VPN they are connected to. We measure the feasibility of connection inference, the consistency of background DNS traffic, DNS-cache behavior, and the ability to precompute and selectively drop tunneled DNS replies on the following operating systems: Ubuntu 22.04 and 24.04, Fedora 42, Debian 13, Windows 10 and 11, macOS Sequoia, and Android 15. We also examine this behavior using two popular browsers, Google Chrome 131.0.x and Firefox 137.0.2.

For the attacker, we created another Ubuntu 22.04 VM on the same research network. This node did not require VPN software, but it used the *farpd* package to place itself in-path between the VPN client and server. This Mallory node also required the packet-crafting and spoofing libraries installed to run the connection inference and injection scripts. In our attack scripts, we use the *libtins* C++ library to craft and spoof custom packets [1].

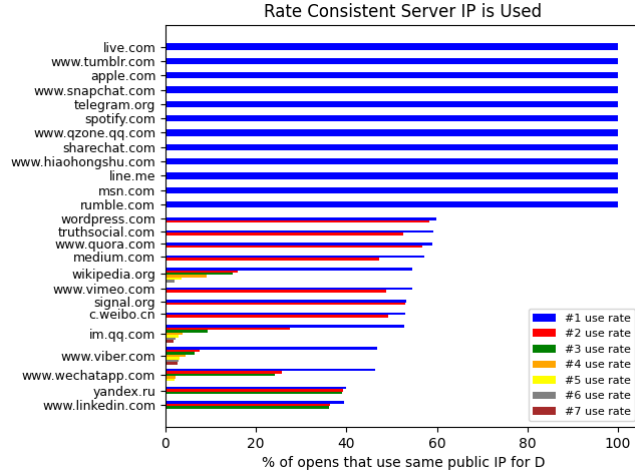


Figure 3: Top 25 domains with the most consistent server IP-address usage from the measured set S of 50 social-media and news domains.

5.2 Consistent IP Usage by SNI

To infer an active connection to a website, DNS server, or application, Mallory needs to determine which IP addresses to include in scan list S because they are most likely to be contacted by the target. When inferring a DNS flow, Mallory will only need to send spoofed packets using the addresses of a small set of widely used DNS servers to confirm which one is being used by the VPN client. Typically, VPN servers will allow clients to retain their existing DNS configuration, such as Google Public DNS at 8.8.8.8 or Quad9 at 9.9.9.9. Alternatively, some commercial VPNs and manually configured deployments require clients to use provider-specific DNS servers. NordVPN also operates several provider-specific DNS server IP addresses. For DNS flows, we assume a candidate set of at most 30 DNS servers. Once Mallory confirms a DNS flow, it identifies the DNS server in use.

For long-lived connections used to browse a social-media feed, TCP or UDP on port 443 is used for the duration of the connection. Websites differ in how they select the public server IP addresses used for browser connections. For Mallory, it is important for the speed and performance of the attack to scan only the IP addresses that have a high likelihood of being used to connect to one of the websites in the scan list S . As shown in Figure 3, major platforms vary significantly in their use of server IP addresses.

In order to test how consistently each of the 50 websites we selected for S selects IP addresses during a web browser connection, we sampled from each of the 110 countries in which NordVPN operates. We wrote a script to open each social site in a web browser five different times, using a different VPN within that country for each round. The collection script forced the measuring system to use one of 20 widely used DNS resolvers worldwide. The results in Figure 3 show that many of the most popular websites, including `x.com`, `telegram.org`, and `snapchat.com` use a small set of server IP addresses, with individual addresses appearing in more than 25% of measured connections. Additionally, even the most heavily

Min IP use %	# Domains	# IPs in S	Success rate	Ave time to hit	Ave scan time
90	11	11	98 / 100	5.01 s	25.57 s
50	16	22	96 / 100	8.12 s	40.97 s
30	23	39	91 / 100	13.38 s	79.99 s
20	32	60	77 / 100	22.84 s	116.92 s
10	44	117	53 / 100	30.41 s	223.34 s
5	48	153	45 / 100	44.12 s	291.49 s

Table 1: Website confirmation attack success rate with different scan lists S

load-balanced websites, such as `facebook.com` and `tiktok.com` still consistently choose between 10–20 IP addresses, each of which is used in approximately 5–10% of measured connections.

These results show that Mallory only needs to include a relatively small set of high-probability server IP addresses for each monitored domain to achieve high connection-confirmation success rates. For many services, only a handful of server IP addresses account for a large fraction of observed connections, while more heavily load-balanced services require modestly larger scan lists. For X, Telegram, and Snapchat, Mallory needs to include fewer than four IP addresses for each. However, for some popular, well load-balanced websites like Facebook and TikTok, Mallory includes each of at least 10 IP addresses with the highest likelihood of being chosen. To increase the likelihood of success, Mallory orders the IP addresses in S from highest to lowest observed use rate. The use rate of an IP, s , is the percentage of measurements that s is connected to when accessing that domain D in the browser. The next section shows how the minimum use rate and number of IP addresses included in S affects the connection-inference success rate.

5.3 Connection Inference Results

Using the experimental setup detailed in Section 5.1, we test the success rate and time required for the attacker to confirm an active connection from the VPN client to one of the 50 sites. In each experiment, the victim VPN client is actively connected to one of our two test VPN servers. To simulate a real attack scenario, the client opens a browser and connects at random to one of N potential websites for a maximum of five minutes per test. We choose N based on the minimum use-rate threshold M for the most commonly used IP address associated with each site. We evaluate the attack script with M set to 90%, 50%, 30%, 20%, 10%, and 5% using the monitored list of website IP addresses. After observing a burst of encrypted traffic, the attacker starts the connection-confirmation attack script. A *burst* is defined as a sum of observed IP packet lengths greater than 7,500 bytes from the VPN client IP address to the VPN server IP address within the preceding two seconds. Once a *burst* is detected, the attacker’s goal is to determine which of the N websites the VPN client is connected to before the connection expires in the VPN server’s conntrack table.

Table 1 highlights a direct trade-off between scan-list coverage and attack success: as coverage expands from 11 to 48 of the 50 target domains, the blind attacker’s success rate falls from 98% to 45%. This is because, as the IP-address scan list grows beyond 50 entries, it becomes increasingly difficult for the attacker to probe the entire ephemeral-port range for each additional IP address within the five-minute window.

For each IP address the attacker scans for an active connection, it may need to try every possible ephemeral port. In our script, running on a default Ubuntu 22.04 VM with spoofing capabilities, the attacker can scan each IP address by sending probes to all possible ports in under 2.5 seconds on average. Because our experiment allows the client to remain connected to the randomly selected website for a maximum of five minutes, the success rate decreases significantly when the attacker has more than 100 IP addresses to scan. At an average of 2.5 seconds per IP address, the attacker does not have enough time to scan every address in the list. Therefore, in our experiments, this attack method is constrained by the time required to scan a single IP address while maintaining negligible packet loss.

We simulate realistic packet loss and delay by placing the VPN servers, VPN client, and attacker VMs in different regions of the country using our VPS infrastructure. When the attacker sends a spoofed probe packet, the packet travels from the attacker VM on our research network at ASU to our test VPN server hosted on a VPS in Dallas. It is then translated through NAT and forwarded through the encrypted tunnel to the client at ASU in Tempe.

5.3.1 Success Factors. In our experiments, we found that the blind attacker's success rate depends on two important factors for each website included in scan list S .

IP use rate: As discussed in Section 5.2, most of the 50 websites we analyzed repeatedly use a small set of IP addresses that the attacker must scan. We found that 39 of the 50 websites use their most frequently observed IP address in at least 20% of measured connections, and 21 of the 50 websites use their most frequently observed IP address in more than 50% of measured connections. The scan list S orders the IP addresses from highest to lowest use rate to increase the likelihood of a "hit" within the five-minute window.

Connection duration and type: The second important factor is how long the target VPN client's connections remain active in the VPN server's conntrack table. We found that only two websites in our list of 50, baidu.com and x.com, immediately close TCP connections to the IP addresses associated with their respective domains. The scan list S places the IP addresses for these two domains first to increase the success rate.

Initially, we assumed that each of the 50 websites in our measurement list would use TCP for its primary web-server connection. However, we discovered that many sites, primarily video-based platforms, rely on UDP connections to their server IP addresses that remain active in the background for several minutes. In our measurements, websites including Snapchat, Spotify, Vimeo, and Discord used QUIC over UDP as the primary protocol for browser connections. Mallory adds a flag to each scan-list entry corresponding to a UDP flow so that the attack script uses the appropriate transport protocol during the scan.

5.4 Tunneled DNS Poisoning for Domain D

The attacker, Mallory, must first choose a target domain D for DNS injection. Ideally, D should be a domain that the victim system is likely to query again in the near future. This decision can be

guided by reconnaissance data, such as the victim's active connections, operating system, or browser, identified using the connection-confirmation attack described earlier.

In our testing, only systems using systemd-resolved cached CNAME records together with their associated A or AAAA records for the duration of the DNS TTL. As a result, domains referenced in a CNAME response could subsequently resolve directly from the cache. In contrast, Windows and macOS cached the CNAME only under the originally queried domain and required a new DNS lookup when querying one of the referenced CNAME domains.

Against Linux targets using systemd-resolved, Mallory can inject a DNS response containing a CNAME mapping from D_1 to an attacker-controlled domain D_2 , along with A and AAAA records for D_2 . This allows Mallory to resolve D_1 to malicious IPv4 or IPv6 addresses of its choosing.

Using a background DNS query provides a significant advantage because Mallory does not need to identify an actively used domain that will soon trigger another DNS request. Instead, the attacker can rely on the repeated system lookup as a continuous injection opportunity, allowing arbitrary malicious mappings for domains intended to leak traffic outside the VPN tunnel.

It is important to note that, in our testing, only standard Linux systems using systemd-resolved both accepted brute-forced DNS replies and reused cached CNAME mappings for subsequent lookups of the referenced domains. As a result, only Linux distributions such as Ubuntu and Fedora were vulnerable to the complete blind injection attack using a background domain as the injection window. Nevertheless, we include results for all evaluated operating systems in Table 2 and Section 5.4.5 to provide a comprehensive comparison of resolver behavior.

5.4.1 Choice of Inject Domain D . The flexibility for Mallory to choose any DNS request that is consistently sent by the victim's device, such as play.google.com, and point it to any arbitrary target domain (e.g., x.com) significantly increases the success rate and danger of this attack. In Table 2, we list the DNS query sent most often by each client operating system. We measured each default operating system and browser listed in the table below while idle for several days with no other processes open to determine which DNS requests were consistently sent in the background and how often. We also measured the maximum TTL for which each system would cache a response if the attacker set the TTL in the DNS reply probes to an extremely large value, such as 604,800 seconds (seven days).

According to our measurements, each operating system consistently queries at least one domain every ten minutes. Because Mallory can drop specific DNS reply flows, the behavior and timing of the DNS retransmissions allow the attacker to identify which operating system is being used. For example, if only four total replies are dropped at two-second intervals, then Mallory knows that the target is likely using Windows.

The two browsers we analyzed query their primary background servers only every 30 minutes and each time they are opened, but they also query the underlying operating system for each website open in the browser every 60 or 120 seconds at most. One clear choice for Mallory is to determine the operating system of the VPN target and then use its most frequently queried background

Target System	Most consistent DNS query (idle)	Ave time between queries	No-reply Duration	Max DNS TTL cached	Accepts 65k txIDs
Ubuntu 22/24+Linux Mint 22.1	connectivity-check.ubuntu.com	150.0 secs	Query every 5s w 24 retries = 2 mins	7200 secs (2 hours)	Yes
Fedora Linux 42	fedoraproject.org	300.0 secs	Query every 5s w 24 retries = 2 mins	7200 secs	Yes
Debian GNU / Linux 13	NA	NA	Query every 5s w 2 retries = 15 secs	No cache by default	Yes
Windows 10/11	v10.events.data.microsoft.com	416.95 sec	Query every 2s w 3 retries = 8s	86,400 secs (24 hours)	5 per query
macOS (Sequoia)	primary-dns-server	989.59 sec	Query every 30s or 90s	604,800 secs (7 days)	1 per query
Android (15)	www.google.com	141.54 secs	Query every 3s w 1 retry = 6s	NA / Until newer reply	Yes
Google Chrome	clientservices.googleapis.com	open + 1800 secs	Depends on OS	60 secs then ask OS	Depends on OS
Firefox	mozilla.cloudflare-dns.com	open + 1410.2 secs	Depends on OS	120 secs then ask OS	Depends on OS

Table 2: Most consistent DNS background queries made by idle OS and browsers.

domain, shown in Table 2, as the injection domain D . With the default operating-system configurations on many Linux desktops using `systemd-resolved`, one successful injection into the VPN client could remain in the system’s cache for up to two hours.

5.4.2 DNS Injection Results. To test the success rate, timing factors, and real-world practicality of the full tunneled DNS injection, we used the same test environment described in Section 5.1 to attempt to inject a malicious DNS entry into the VPN target’s operating system. In our experiments, we found that Windows 10 and 11 close the source port that sent the DNS query after the fifth incorrect transaction ID is received. This Windows resolver behavior was added after researchers used brute-force DNS injection to poison the operating-system cache in 2022 [2]. macOS Sequoia retransmits the DNS query over TCP if a single incorrect transaction ID is received. Only Ubuntu 24.04, Fedora 42, Linux Mint 22.1, Debian 13, and Android 15 did not close the source UDP port after receiving thousands of incorrect transaction IDs. However, Debian does not have `systemd-resolved` enabled by default, so it does not cache the successful injection unless a browser is open, which generally has a maximum TTL of only one or two minutes.

We conducted measurements on each major operating system to determine the most frequently queried background domain, which would provide the most opportunities for Mallory to inject a DNS reply. Using the predictable DNS-query findings shown in Table 2, we tested the DNS-injection attack 1,000 times against the most consistently queried domain on a default Ubuntu 24.04 desktop: `connectivity-check.ubuntu.com`.

5.4.3 Success Factors. Mallory uses the function E and samples from the target domain D to determine the sizes of packets to drop so that the VPN client never receives the legitimate DNS replies. We used our Python script, which takes any domain D , repeatedly queries it, and determines the required iptables rules. Each resolver service handles DNS retransmissions differently, with a configurable timeout period and maximum number of retransmissions. `systemd-resolved`, used by default on Ubuntu desktops, has a default five-second timeout and retransmits the query with the same source port and transaction ID 24 times, or for two minutes, during its “lookup loop.”

Most of the systems in Table 2 give the attacker another opportunity to inject within the next few minutes because the most common domain is queried so frequently. Therefore, if the attacker fails, they will have another opportunity within minutes. On some operating systems, such as Ubuntu, the DNS lookups appear to be sent continuously every five seconds when the drop rule is in place

for D , because the system queries repeatedly for two minutes and then starts a new query during the third minute.

The most important factors affecting the likelihood that the attacker successfully injects the malicious DNS reply are whether the operating system accepts brute-forced transaction IDs and how long the VPN client keeps open the UDP source port that sent the request while awaiting a reply. If the client closes the source port quickly, e.g., within five seconds, the attacker is unlikely to infer the port and spoof each of the 65,536 possible transaction IDs in time. As shown in Table 2, most tested Linux operating systems and DNS resolver services, such as `systemd-resolved` on Ubuntu 24.04, keep the port open for 120 seconds while resending the same DNS query every five seconds with the same transaction ID. Typically, the client operating system does not need to retransmit before receiving a reply from the primary or fallback DNS servers, but Mallory drops each legitimate reply packet in this attack.

The average time for Mallory to complete the port inference and then brute-force the DNS injection in our experiments was 9.82 seconds. The longer time is due to the target often having multiple DNS lookup flows active simultaneously, into which Mallory attempts to inject the reply for D . The full injection attempt was successful 92.9% of the time against an Ubuntu 24.04 client. The success rate drops to approximately 0.00763% on Windows 10 and 11 because only five of the 65,536 brute-forced transaction IDs are accepted before the client closes the port and retries the DNS query on a new port.

5.4.4 Injected Reply Details. Mallory can set the TTL of the injected DNS reply to force the client’s operating system to cache and use the response for as long as possible. Most modern browsers, including Firefox and Chrome, limit the DNS TTL to 60 or 120 seconds. However, this has no effect on the maximum TTL cached by the underlying operating-system resolver. Therefore, when the TTL for browser domain D expires, the browser simply asks the operating-system resolver for the mapping to D , which may remain cached for hours, with a maximum of two hours when using `systemd-resolved`.

On operating systems with `systemd-resolved` enabled, the service caches up to 10 domains included in a DNS response containing CNAME records. Therefore, Mallory can redirect up to 10 domains to an arbitrary IP address of their choosing with one successful injection that remains cached and in use for two hours. The attacker can choose to redirect these injected domains to one of their own malicious IP addresses. Alternatively, they can map the domains to the public IP address of the VPN server, which forces the outgoing packets to bypass the VPN tunnel encryption altogether because

of the outgoing packet-routing rules. The attacker can then view the packets as though no VPN were being used and use DNAT to redirect the packets to the intended server IP address and observe the SNI for any of the 10 injected domains in cleartext.

5.4.5 Results vs. Non-Linux Targets. macOS side channel: When macOS devices receive a DNS reply for a domain D that matches the lookup port used for D but contains an incorrect transaction ID, the mDNSResponder daemon immediately resends the lookup over TCP and bypasses Mallory's packet-drop rules. However, if the macOS device receives a DNS reply containing a domain other than D on the port used for the original lookup, with any transaction ID, it continues retransmitting the lookup for D until it receives a DNS reply that includes D as the query field. Therefore, Mallory can use this side channel to confirm that domain D is being queried by the macOS device by sending replies containing different domains and observing whether the device continues retransmitting the lookup. If the injected DNS reply contains the correct domain for the lookup flow but an invalid transaction ID, the device immediately closes the flow and retries over TCP. Otherwise, it continues retransmitting the lookup every 30 or 90 seconds for 10 retries.

Windows 10 and 11: Because Windows devices close the source port used for the DNS query for D after five incorrect transaction IDs, regardless of the domain in the reply's query field, the attacker has only a 5 in 65,536 chance of a successful injection, assuming that the correct source port and domain D were inferred. Mallory can still use the drop rules for DNS replies for D to impose a DoS condition on a VPN client attempting to access a service hosted by D . This allows for a more fine-grained form of DoS than simply dropping packets from the VPN connection because it can block specific services within the tunnel.

Android 15: Like Linux devices, Android accepts brute-forced transaction IDs. Although the device has the shortest timeout and only one retransmission per query, when an injected DNS reply is accepted, it honors the TTL for D until a more recent DNS query for D succeeds. Therefore, as long as Mallory keeps the drop rules for D in place, the injected mapping remains in the netd cache. We verified that even after using the Clear host cache button on the Chrome DNS page, we still received the cached injected mapping for D while the drop rules remained in place. The cached entry was cleared only after switching or restarting networks. If the drop rules are removed and a legitimate DNS reply is received for D , it overwrites the cached entry. Thus, Android is also susceptible to DNS cache poisoning, but the success rate is substantially lower. Because mobile devices require manual testing, we did not measure this lower rate.

6 Does traffic obfuscation mitigate this attack?

In this section we evaluate the case that there is a total equivocation of packet size in the encrypted tunnel. Our attacks described in this paper implicitly use both the encrypted packet size and its timing as a combined channel to infer the ephemeral port of the DNS request. What if, through padding or some other mechanism, the size component of the channel is completely taken away from the attacker?

More broadly, mitigation strategies against VPN blockage [10, 45, 47] or website/webpage fingerprinting [13, 20, 43] often use

padding or packet delays to hide side channels in packet size and timing. How robust is the signal that we use for our attacks against this?

From an information-theoretic perspective, the attacker's goal is to learn information about the ephemeral port number as quickly as possible. Thus, the function we want to maximize for any round of guessing ephemeral port numbers is the information gained divided by the amount of time taken. Suppose the attacker has a confidence range with a minimum (a) and maximum (b) value the round-trip time could be between the attacker machine and the VPN server. Let r , for rate, be the reciprocal of the total amount of time it takes for the attacker to guess (by spoofing a packet with) every possible ephemeral port number within the set of candidates still left in any round. Let x be the fraction of this total time that the attacker's confidence interval for round trip time falls within, so $x = \frac{b-a}{r}$. Clearly, if the attacker sends at too high a rate all (or most) of the possible ephemeral port numbers will fall within the confidence interval and the attacker will learn nothing (or almost nothing). As the rate, r , approaches 0 exactly one possible ephemeral port number will fall within the confidence interval, but it will take an extremely long time to learn that information. The question is, what is the optimal rate the attacker should send packets at in each round to narrow down the exact ephemeral port number they seek?

Because x is a fraction that is normalized to the length of the confidence interval $b - a$, the continuous information, we can also normalize the amount of differential entropy that exists in the entire round of the attacker guessing every possible ephemeral port number to be $\log_2(1)$ and the differential entropy for the ephemeral port guesses that will fall within the confidence interval to be $\log_2(x)$. We can also normalize the rate packets are sent at to be simply $\frac{1}{x}$. So, dividing the information gained in a round by the rate we obtain:

$$f(x) = x(\log_2(1) - \log_2(x))$$

To maximize this expression for the rate of information learned we take the derivative:

$$f'(x) = \frac{d}{dx} \left(-\frac{x \cdot \ln(x)}{\ln(2)} \right) = -\frac{1 + \ln(x)}{\ln(2)}$$

Solving for $f'(x) = 0$, we obtain a maximum at:

$$x = \frac{1}{e} \approx 0.36787944$$

That is, to learn the most information in the least amount of time, the attacker should, in each round, send ephemeral port guesses at a rate such that $\frac{1}{e}$ of the guesses will fall within the confidence interval $b - a$ and the rest outside this interval. For example, if the attacker has $2^9 = 512$ port numbers left to explore and the confidence interval between the minimum and maximum round trip times is 20ms, the rate should be $\approx 9,417$ packets per second.

A more realistic and conservative assumption for the round trip time using a Gaussian distribution does not change the attacker's send rate significantly. We leave the details for this case of a Gaussian distribution approximation of the round trip time for the appendix in section B.2.

7 Defense Modules

We present installable `connttrack` modules that can be deployed on a VPN server to defend against the attacks demonstrated in this paper. These modules protect against both brute-force application-layer injection attacks, such as DNS injection, and connection-inference attacks. They analyze transport- and application-layer protocol information to ensure that only packets consistent with the expected connection state are forwarded to the VPN client. The source code for the custom modules is included in the accompanying artifact [24].

The TCP defense is comparatively straightforward because it can use transport-layer information, such as TCP flags indicating repeated SYN-ACK packets, to identify spoofed traffic. In contrast, defenses for UDP-based protocols cannot rely on transport-layer state and must instead analyze application-layer semantics to distinguish legitimate packets from attacker-generated spoofed packets.

We do not evaluate the runtime performance of the proposed `connttrack` modules because a meaningful benchmark would require reproducing the traffic patterns and connection state of a production VPN server supporting hundreds of concurrent clients. Such a deployment-scale evaluation is beyond the scope of this work.

7.1 TCP Connection Inference Defense

The TCP connection inference attack exploits the behavior of `connttrack` on modern VPN servers, which will forward TCP SYN-ACK packets that match an existing connection 5-tuple in the `connttrack` table. As described in Section 4.3, Mallory spoofs arbitrary SYN-ACK packets to the VPN server, which are then forwarded to the target VPN client if they correspond to an active connection. Since only a single probe packet is required per candidate port, the attack can efficiently enumerate active TCP connections associated with a target server address.

Linux systems include the `nf_connttrack_tcp_be_liberal` kernel parameter, which determines whether out-of-window TCP packets are treated as `INVALID`. By default, this parameter is disabled (`0`), causing `connttrack` to drop TCP packets that do not contain valid in-window sequence numbers. However, SYN-ACK packets bypass this validation because they are interpreted as part of the TCP connection establishment process. In contrast, other packet types, such as PUSH-ACK packets, are dropped unless they contain valid sequence and acknowledgment values corresponding to the active connection state.

Mallory exploits this SYN-ACK handling behavior to avoid guessing valid 32-bit TCP sequence and acknowledgment values for each probe. Consequently, the attacker only needs to identify the correct connection 5-tuple in order to infer active TCP sessions.

The VPN server can mitigate this attack using a simple `iptables` rule that drops SYN-ACK packets matching connections already in the `ESTABLISHED` state.

```
iptables -I FORWARD -p tcp --tcp-flags SYN,ACK SYN,ACK \
-m connttrack --ctstate ESTABLISHED -j DROP
```

This prevents attacker-generated probes from being forwarded to VPN clients and eliminates the ability to infer active TCP connections through packet sniffing.

While this mitigation is effective for TCP inference attacks that rely on bypassing TCP sequence and acknowledgment validation, a comparable defense is substantially more difficult for UDP-based protocols such as DNS. UDP does not include transport-layer sequence numbers or control flags that distinguish legitimate packets from attacker-generated probes. As a result, the module must rely on application layer information depending on the protocol, such as the transaction ID in DNS traffic.

7.2 DNS Injection Defense Module

The attack takes advantage of the fact that the tested Linux operating systems allow unlimited DNS replies with incorrect transaction IDs to reach the UDP port that sent the query. Therefore, the attacker can spoof replies containing all 65,536 possible transaction IDs until one is accepted. The victim's use of a VPN does not prevent the attack because the attacker spoofs DNS replies into the tunnel through an inferred DNS connection flow on the VPN server.

Furthermore, even on devices that reject brute-forced DNS replies after several incorrect attempts, such as macOS and Windows 11, the attacker can still infer the 5-tuple used for the query. Because the attacker can send empty or arbitrary UDP packets that are forwarded to the VPN client without validation by the VPN server, the middlebox can infer connections and identify the DNS server in use.

To address this issue, we developed a `connttrack` module that can be installed on a Linux device to protect against this attack [23]. Every outgoing DNS query creates a `connttrack` entry to which the module attaches per-flow state that tracks the outgoing transaction ID and the number of incoming replies with incorrect IDs. The module increments a counter on the DNS entry each time it receives a reply with a transaction ID that does not match the original query. Once the counter exceeds a configurable threshold (10 by default), the module tears down the `connttrack` entry so that no additional attacker packets are forwarded. Therefore, the defense module works similarly to the default handling on Windows systems, which close the port after five incorrect IDs. Additionally, after a transaction ID is assigned to an outgoing DNS flow, all invalid, short, or empty UDP packets arriving on that flow are dropped.

We use the attacker's 92.9% success rate when identifying the DNS connection flow and attempting to inject a valid DNS reply using all 65,536 IDs as the baseline without the defense. This module would reduce the attacker's success rate to approximately 0.0153%, because the attacker would have only a 10/65,536 chance of guessing the correct ID for each port used by a query.

7.3 QUIC Defense Module

HTTP/3 runs exclusively over QUIC [18]. According to 2026 statistics, approximately 34% of the top global websites support HTTP/3 using QUIC [33]. In our measurements, we found that 8 of the 50 websites analyzed consistently use the protocol. Because these websites do not typically rely on standard TCP connections, the defense module in Section 7.1 does not protect against the connection-inference attack. Therefore, we developed a `connttrack` defense module that first confirms that an outgoing flow uses QUIC and then ensures that only valid packets are forwarded to the VPN client.

To prevent empty or arbitrary UDP packets from reaching the VPN client, the module tracks the connection ID field, similar to how it tracks transaction IDs for the DNS defense. It closes flows that receive multiple packets with incorrect connection IDs or invalid, undersized, or empty payloads. QUIC requires the module to track additional information. Handshake packets carry an explicit connection ID length, but later packets do not, so the module must first learn this length from the handshake. The module also inspects the first packet of each new flow to confirm that it is QUIC before attaching and enforces a minimum payload size on later packets so that empty or truncated packets are dropped before reaching the QUIC stack.

7.4 DTLS Defense Module

DTLS (Datagram Transport Layer Security) is the UDP equivalent of TLS over TCP. It has primarily been adopted by applications using VoIP, WebRTC, and many IoT devices. Exact usage statistics are difficult to estimate because many of these applications use TLS or QUIC alongside DTLS for voice- or video-based traffic. Therefore, we include a defense module for DTLS so that the primary protocols observed in practice are protected against the connection-inference attack.

The DTLS defense module works similarly to the QUIC defense module. After confirming that a flow is DTLS by checking its protocol identifiers and record structure during the initial handshake, it tracks the Connection ID (CID; RFC 9146 [34]) carried in each record for that flow. Once the module observes multiple invalid packets, whether from incorrect CIDs or arbitrary or empty data, it closes the connection flow. This prevents an attacker from using spoofed packets to probe for and confirm active 5-tuples belonging to a DTLS-based service.

8 Related Work

Several studies examine passive network attacks that use ML-based classifiers to fingerprint websites through encrypted channels such as VPNs and are subject to high false-positive rates [15, 32, 36, 37]. Other work examines traffic leaks caused by VPN misconfiguration [16, 21, 49]. In 2021, Tolley et al. [39] showed that active attackers could exploit the weak host model client-side and VPN server NAT functionality to interact with connections inside the VPN tunnel. We extend this work by examining server-side attacks across a large set of websites and the practicality of DNS injection on modern operating systems. In 2024, Mixon-Baca et al. [30] showed that an attacker acting as another VPN client can use IP and port collisions on the VPN server to escalate from an off-path to an in-path attacker using the port-shadow attack.

Xue et al. [48] showed that an active on-path attacker could spoof the plaintext DNS reply for the VPN server's domain to reroute traffic outside the encrypted tunnel. More recently, researchers combined passive filtering and active probing to fingerprint OpenVPN connection flows on a million-user ISP with an extremely low false-positive rate [47]. Our work focuses on a targeted VPN client-to-server flow and examines what an active attacker can confirm about the user's connections, the potential to escalate to DNS injection, and defenses at the VPN server.

9 Discussion

The attack exploits a protocol-level issue in modern remote-access VPNs: an attacker can infer the presence of an active connection because VPN servers behave differently when a spoofed packet matches an existing transport-layer connection than when it does not. VPN servers use NAT-based connection-tracking services (e.g., `conntrack` on Linux) to map external packets to internal VPN client flows, forwarding any packet that matches an existing connection tuple to the associated client. Application-layer proxy protocols such as Shadowsocks are immune to this: they authenticate and process traffic at the application layer before relaying it, rather than exposing the transparent IP-forwarding abstraction used by VPN protocols such as OpenVPN and WireGuard, so spoofed packets are discarded before reaching the proxied connection.

The primary limitation of the DNS-injection vulnerability is that recent versions of Windows and macOS already stop accepting DNS replies after several responses with incorrect transaction IDs. Mitigating this attack requires the resolver to maintain additional state and validate repeated transaction-ID mismatches. Since local DNS resolver services such as `systemd-resolved` are provided by Linux distributions rather than the kernel, implementing this defense there instead of as a kernel module avoids added kernel complexity while preserving deployment flexibility.

All tested operating systems remain vulnerable to connection inference because the underlying issue is the VPN server's 5-tuple-based NAT matching, which does not validate incoming external packets.

10 Conclusion

The blind in-path attacks presented in this paper require a dedicated middlebox attacker able to actively spoof and selectively drop packets, but we show that common network attacks such as DNS injection and connection confirmation can still be performed reliably. The attacker can use precalculated packet sizes to create packet-drop rules that open injection windows for malicious DNS replies in the client's DNS cache. To mitigate these attacks, we developed lightweight defense modules that prevent the packet-forwarding and DNS-injection behaviors exploited by the attacker while preserving compatibility with existing VPN deployments.

Under the same threat model, a client not connected to a VPN would remain vulnerable to these attacks and could be compromised with only a small number of spoofed packets. Although the encrypted tunnel provides a substantial layer of protection, users should be aware that the attacks presented in this paper remain possible against current VPN architectures if an attacker is willing to expend sufficient effort. For digital-rights activists, journalists, and other high-risk users, application-layer proxy systems such as Shadowsocks, anonymity networks such as Tor, or security-focused operating systems such as Tails provide stronger resistance to the forwarding behavior exploited by our attacks.

Acknowledgments

This material is based upon work supported by the National Science Foundation under Grant Number CNS-2452885.

References

- [1] 2025. libtins: packet crafting and sniffing library C++. <https://libtins.github.io/download/>
- [2] Fatemah Alharbi, Yuchen Zhou, Feng Qian, Zhiyun Qian, and Nael Abu-Ghazaleh. 2022. Dns poisoning of operating system caches: Attacks and mitigations. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2022), 2851–2863.
- [3] Simurgh Aryan, Homa Aryan, and J Alex Halderman. 2013. Internet censorship in Iran: A first look. In *3rd USENIX Workshop on Free and Open Communications on the Internet (FOCI 13)*.
- [4] Bnamerica. 2023. 92 million Brazilians access the internet exclusively via mobile phone. <https://www.bnamerica.com/en/news/92-million-brazilians-access-the-internet-exclusively-via-mobile-phone>
- [5] Danilo Bruschi, Andrea Di Pasquale, Andrea Lanzi, and Elena Pagani. 2024. Ensuring cybersecurity for industrial networks: A solution for ARP-based MITM attacks. *Journal of Computer Security* 32, 5 (2024), 447–475.
- [6] Jamon Camisso. 2022. How To Set Up WireGuard on Ubuntu 22.04. <https://www.digitalocean.com/community/tutorials/how-to-set-up-wireguard-on-ubuntu-22-04> Accessed: 2025-08-26.
- [7] Jamon Camisso and Mark Drake. 2022. How To Set Up and Configure an OpenVPN Server on Ubuntu 22.04. <https://www.digitalocean.com/community/tutorials/how-to-set-up-and-configure-an-openvpn-server-on-ubuntu-22-04> Accessed: 2025-08-26.
- [8] Zimo Chai, Amirhossein Ghafari, and Amir Houmansadr. 2019. On the importance of {Encrypted-SNI}{({({({ESNI})})})})} to censorship circumvention. In *9th USENIX Workshop on Free and Open Communications on the Internet (FOCI 19)*.
- [9] Sonali Chandel, Zang Jingji, Yu Yunnan, Sun Jingyao, and Zhang Zhipeng. 2019. The golden shield project of china: A decade later—an in-depth study of the great firewall. In *2019 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC)*. IEEE, 111–119.
- [10] Kevin P Dyer, Scott E Coull, and Thomas Shrimpton. 2015. Marionette: A programmable network traffic obfuscation system. In *24th USENIX Security Symposium (USENIX Security 15)*. 367–382.
- [11] Roya Ensafi, Philipp Winter, Abdullah Mueen, and Jedidiah R Crandall. 2015. Analyzing the Great Firewall of China over space and time. *Proceedings on privacy enhancing technologies* (2015).
- [12] VPN Gate. 2025. VPN Gate User Countries Realtime Ranking. <https://www.vpngate.net/en/region.aspx>
- [13] Jiajun Gong and Tao Wang. 2020. Zero-delay lightweight defenses against website fingerprinting. In *29th USENIX Security Symposium (USENIX Security 20)*. 717–734.
- [14] Alireza Hashemzadegan and Ali Gholami. 2022. Internet Censorship in Iran: An Inside Look. *Journal of Cyberspace Studies* 6, 2 (2022), 183–204.
- [15] Dominik Herrmann, Rolf Wendolsky, and Hannes Federrath. 2009. Website fingerprinting: attacking popular privacy enhancing technologies with the multinomial naïve-bayes classifier. In *Proceedings of the 2009 ACM workshop on Cloud computing security*. 31–42.
- [16] Muhammad Ikram, Narseo Vallina-Rodriguez, Suranga Seneviratne, Mohamed Ali Kaafar, and Vern Paxson. 2016. An analysis of the privacy and security risks of android vpn permission-enabled apps. In *Proceedings of the 2016 internet measurement conference*. 349–364.
- [17] FOX IT. 2015. Deep dive into QUANTUM INSERT. <https://blog.fox-it.com/2015/04/20/deep-dive-into-quantum-insert/>
- [18] Jana Iyengar and Martin Thomson. 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000. doi:10.17487/RFC9000
- [19] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Rachel Greenstadt. 2014. A critical evaluation of website fingerprinting attacks. In *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*. 263–274.
- [20] Marc Juarez, Mohsen Imani, Mike Perry, Claudia Diaz, and Matthew Wright. 2016. Toward an efficient website fingerprinting defense. In *European Symposium on Research in Computer Security*. Springer, 27–46.
- [21] Mohammad Taha Khan, Joe DeBlasio, Geoffrey M Voelker, Alex C Snoeren, Chris Kanich, and Narseo Vallina-Rodriguez. 2018. An empirical analysis of the commercial vpn ecosystem. In *Proceedings of the internet measurement conference 2018*. 443–456.
- [22] Suman Khurana. 2015. A security approach to prevent ARP poisoning and defensive tools. *International Journal of Computer and Communication System Engineering* 2, 3 (2015), 431–437.
- [23] Beau Kujath. 2026. DNS Defense Kernel Module Linux. <https://doi.org/10.5281/zenodo.18238230>
- [24] Beau Kujath. 2026. MITT Research Artifact. <https://doi.org/10.5281/zenodo.22754386>
- [25] Bill Marczak, Jakub Dalek, Sarah McKune, Adam Senft, John Scott-Railton, and Ron Deibert. 2018. BAD TRAFFIC: Sandvine’s PacketLogic Devices Used to Deploy Government Spyware in Turkey and Redirect Egyptian Users to Affiliate Ads? <https://citizenlab.ca/2018/03/bad-traffic-sandvines-packetlogic-devices-deploy-government-spyware-turkey-syria/>. Citizen Lab report.
- [26] Bill Marczak, Nicholas Weaver, Jakub Dalek, Roya Ensafi, David Fifield, Sarah McKune, John Scott-Railton, Ron Deibert, and Vern Paxson. 2015. An Analysis of China’s “Great Cannon”. In *Proceedings of FOCI 2015 (USENIX Workshop on Free and Open Communications on the Internet)*. <https://www.usenix.org/system/files/conference/foci15/foci15-paper-marczak.pdf>
- [27] Alexander Master and Christina Garman. 2023. A Worldwide View of Nation-state Internet Censorship. *Free and Open Communications on the Internet* (2023).
- [28] Sarah McLaughlin. 2024. Brazil bans X — and threatens daily \$9,000 fine for those still trying to use it. <https://www.thefire.org/news/blogs/free-speech-dispatch/brazil-bans-x-and-threatens-daily-9000-fine-those-still-trying-use> Accessed: 2025-02-3.
- [29] Dasha Milden. 2025. CNET Survey: 47% of Americans Use VPNs for Privacy. That Number Could Rise. Here’s Why. *CNET* (2025). <https://www.cnet.com/tech/services-and-software/vpn-survey-2025/>
- [30] Benjamin Mixon-Baca, Jeffrey Knockel, Diwen Xue, Tarun Ayyagari, Deepak Kapur, Roya Ensafi, and Jedidiah R Crandall. 2024. Attacking connection tracking frameworks as used by virtual private networks. *Proceedings on Privacy Enhancing Technologies* YYYY (X) 1 (2024), 18.
- [31] Gabi Nakibly, Jaime Scholnik, and Yossi Rubin. 2016. {Website-Targeted} False Content Injection by Network Operators. In *25th USENIX Security Symposium (USENIX Security 16)*. 227–244.
- [32] Sanghak Oh, Minwook Lee, Hyunwoo Lee, Elisa Bertino, and Hyoungshick Kim. 2023. AppSniffer: Towards robust mobile app fingerprinting against VPN. In *Proceedings of the ACM Web Conference 2023*. 2318–2328.
- [33] Pasquale Pillitteri. 2026. QUIC Protocol: The Silent Revolution Reshaping the Internet (Made Simple). <https://pasqualepillitteri.it/en/news/2063/quic-protocol-internet-revolution-explained>
- [34] Eric Rescorla, Hannes Tschofenig, Thomas Fossati, and Achim Kraus. 2022. Connection Identifier for DTLS 1.2. RFC 9146. doi:10.17487/RFC9146
- [35] Victor Ronin. 2023. Understanding IPTables, NAT, Contrack and Similar Tools (With Resources to Help). <https://aembit.io/blog/understanding-iptables-nat-contrack-and-similar-tools-with-resources-to-help/>
- [36] Yan Shi and Subir Biswas. 2015. Detecting tunneled video streams using traffic analysis. In *2015 7th International Conference on Communication Systems and Networks (COMSNETS)*. IEEE, 1–8.
- [37] Yan Shi and Subir Biswas. 2016. Protocol-independent identification of encrypted video traffic sources using traffic analysis. In *2016 IEEE International Conference on Communications (ICC)*. IEEE, 1–6.
- [38] statecounter GlobalStats. 2026. Mobile Operating System Market Share Brazil (Jan 2025 - Jan 2026). <https://gs.statcounter.com/os-market-share/mobile/brazil>
- [39] William J. Tolley, Beau Kujath, and Jedidiah R. Crandall. 2021. Blind In/On-Path Attacks and Applications to VPNs. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association. <https://www.usenix.org/system/files/sec21-tolley.pdf>
- [40] Canonical Ubuntu. 2025. CVE-2019-14899. <https://ubuntu.com/security/CVE-2019-14899> Ubuntu update on CVE.
- [41] Mathy Vanhoef. 2023. TunnelCrack: Leaking VPN Traffic by Manipulating Routing Tables. In *Black Hat Europe, Date: 2023/12/06-2023/12/07, Location: London, United Kingdom*.
- [42] Ivana Vojinovic. 2024. VPN Statistics for 2024 – Keeping Your Browsing Habits Private. <https://dataprot.net/statistics/vpn-statistics/> Accessed: 2025-02-03.
- [43] Tao Wang and Ian Goldberg. 2017. {Walkie-Talkie}: An efficient defense against passive website fingerprinting attacks. In *26th USENIX Security Symposium (USENIX Security 17)*. 1375–1390.
- [44] Human Rights Watch. 2025. Russia Clamps Down on Online Searches. <https://www.hrw.org/news/2025/07/24/russia-clamps-down-on-online-searches> Accessed: 2026-05-31.
- [45] Diwen Xue, Michalis Kallitsis, Amir Houmansadr, and Roya Ensafi. 2024. Fingerprinting Obfuscated Proxy Traffic with Encapsulated {TLS} Handshakes. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2689–2706.
- [46] Diwen Xue, Benjamin Mixon-Baca, ValdikSS, Anna Ablove, Beau Kujath, Jedidiah R Crandall, and Roya Ensafi. 2022. TSPU: Russia’s decentralized censorship system. In *Proceedings of the 22nd ACM Internet Measurement Conference*. 179–194.
- [47] Diwen Xue, Reethika Ramesh, Arham Jain, Michaelis Kallitsis, J Alex Halderman, Jedidiah R Crandall, and Roya Ensafi. 2025. OpenVPN is open to VPN fingerprinting. *Commun. ACM* 68, 1 (2025), 79–87.
- [48] Nian Xue, Yashaswi Malla, Zihang Xia, Christina Pöpper, and Mathy Vanhoef. 2023. Bypassing tunnels: Leaking {VPN} client traffic by abusing routing tables. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5719–5736.
- [49] Qi Zhang, Juanru Li, Yuan Yuan Zhang, Hui Wang, and Dawu Gu. 2017. Oh-Pwn-VPN! security analysis of OpenVPN-based Android apps. In *International Conference on Cryptology and Network Security*. Springer, 373–389.

A Open Science

We release our replication repository at the anonymous link [24], with README files describing each component and how to run it. It includes the source code for the full attack scan, an installable Linux kernel defense module, a firewall-rule generator for a given domain, and the scripts used to collect and parse PCAPs. We omit the raw PCAPs themselves (over 500GB) but plan to release the full dataset on publication.

The collection scripts can be run on any Internet-connected machine with root access (a valid NordVPN authentication is required, since the scripts randomly connect to VPN servers worldwide while capturing traffic). The parsing scripts recover consistent DNS usage per test operating system and SNI use rate per public IP address, and regenerate Figure 3.

Main Contributions: the attack script (a C++ program that scans a list of IP addresses for active VPN connections and, in DNS mode, attempts injection into open DNS flows); Linux kernel conntrack defense modules addressing the TCP, DNS, QUIC, and DTLS attacks; a DNS-drop package for determining the iptables rules needed to drop DNS replies for a given domain; the PCAP collection and parsing scripts described above; and the full mapping of observed TLS SNI values to each of the 50 apps/domains in scan list *S*, provided as a CSV and a Markdown table.

B Appendix

B.1 MITT Attack Pre-Reqs

To conduct the active attack, Mallory needs a simple formula for the encrypted packet length E given an initial IP packet length, and must first determine whether the VPN client-server connection uses static or bucket-based padding.

B.1.1 Pre-calculate Any Tunneled Packet Size. The attacker pre-calculates the post-tunnel length $E1$ of any IP packet $IP1$ sent through the tunnel as $E1 = IP1 + p$, where p is the padding added by the VPN tunnel. The value of p depends on whether the VPN uses static or bucket-based padding, and on the transport protocol (UDP or TCP) of $IP1$.

Static protocol VPN target: OpenVPN is a static-length VPN protocol: it does not implement packet length hiding, and by default uses AES-256-GCM or AES-128-GCM as its data channel cipher, giving a fixed encapsulation overhead. The padding value p is therefore constant and can be recovered by subtracting the original packet length from the tunneled length.

In practice, Mallory estimates p by finding the most frequent packet size in the encrypted tunnel over time: most device traffic is TCP, and empty TCP ACKs produce a highly consistent encrypted size, so (Figure 5) one packet length dominates. In our measurements, over 25% of packet lengths observed by the middlebox were encrypted TCP ACKs, giving a reliable baseline for estimating p and inferring the protocol in use.

Since the IP header is 20 bytes, the UDP header 8 bytes, and the TCP header generally 20 bytes, TCP headers add 12 more bytes than UDP; so p for pre-calculated UDP packets is 12 bytes less than for TCP packets of the same data length:

$$E1_{TCP} = IP1 + p, \text{ where } p = 64 \text{ for OVPN}$$

$$E1_{UDP} = IP1 + (p - 12)$$

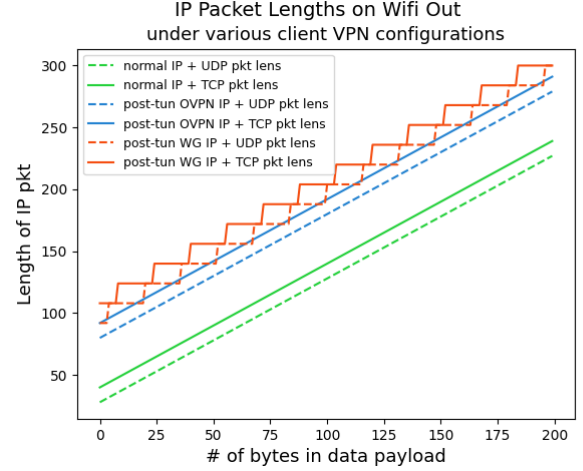


Figure 4: Consistent Size of IP packets as client sends increasing data payloads with TCP/UDP.

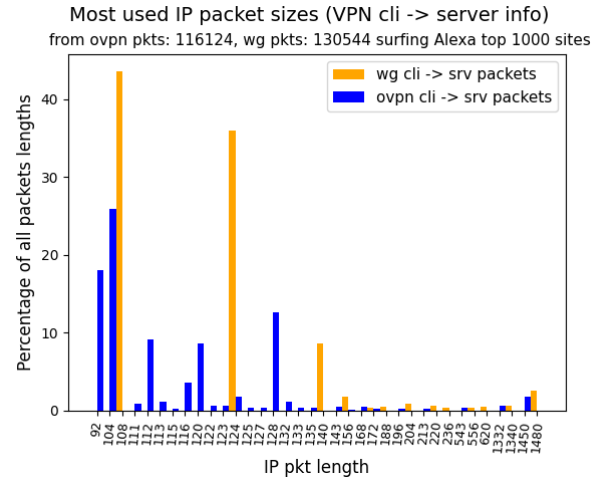


Figure 5: Rate of IP packet length used by OVPN client vs WG client to 1k top websites in browser

Bucket protocol VPN target: WireGuard and NordLynx instead hide packet length by padding into fixed 16-byte buckets: encapsulation overhead is still constant, but the final encrypted length is rounded up to the next 16-byte boundary.

As shown in Figure 4, Mallory models the encrypted length as $(E1) = (p + IP1 + z)$, incrementing z from 0 to 16 until $(E1 - IP1_{hdrlen}) \bmod 16 = 0$, which gives the 16-byte bucket into which $IP1$ falls after encapsulation. $IP1_{hdrlen}$ is the combined IP + transport header length added by the tunnel: 28 bytes for UDP, 40 for TCP.

Appendix B.1.2 describes how Mallory determines the target's VPN protocol via passive traffic analysis; distinguishing static from bucket-based padding is trivial from encrypted packet size distributions alone.

B.1.2 Determining Target VPN Type. In this threat model, the attacker, Mallory, can determine the type of VPN encryption protocol being used by the client in multiple ways. Mallory can use the public IP of the VPN server to perform a reverse DNS lookup to determine if the domain maps to a public OpenVPN server, a NordVPN domain, or any other domain that links to a VPN server.

Alternatively, Mallory could also use traffic sniffing techniques to determine the amount of padding (if any) being used by the encrypted tunnel, and whether or not the encryption scheme being used is *static* or *padded*. For example, if there are no two packets sent between the VPN client and VPN server that have a size difference less than 16 bytes, then Mallory knows it is a padded VPN using 16 byte buckets.

However, if Mallory sniffs encrypted packets being sent between the client where any two IP packets have a difference of 1 byte, then there is no “bucket” padding being used. Furthermore, Mallory can quickly infer the static padding value, p , by sniffing all traffic and counting the packet size that occurs most. In practice as shown in 4, the vast majority of connections made through the VPN will be TCP connections. TCP connections have to send empty acknowledgment (ACK) packets frequently throughout the session to notify the sender that new data was received.

In an empty TCP ACK, the packet will generally be size 40 pre-tunnel from the IP header of length 20 and TCP header of length 20 with no data. Therefore, the packet size post-tunnel that will be seen most if the attacker sniffs and counts all tunneled packet sizes being sent, will be the size of a tunneled empty ACK packet as shown in Figure 5. According to the chart, the encrypted empty TCP packet will be length $104 \rightarrow E = 64 + 40$ for static OVPN, and for Wireguard (with padding) it will be $108 \rightarrow E = (64 + z) + 40$ where z is 4 since $108 - 28 = 80 \bmod 16 = 0$.

Using this E formula, the attacker can now compare the size of the pre-calculated TCP ack packets to the most seen packet size sent from the VPN client to VPN server. Mallory will also use this formula to pre-calculate the size of their own spoof probes post-tunnel and specific tunneled DNS packet sizes that will be dropped in order to create a window to inject a DNS reply.

B.2 Send Rate with Gaussian Approximation for RTT

Assuming a uniform distribution over the entire range of port numbers between the minimum and maximum round trip time is overly conservative, however. In practice, the attacker will know something about the distribution of possible round trip times, and can conservatively approximate this distribution based on observations with a Gaussian distribution. Let σ be the standard deviation of this Gaussian distribution approximation for round trip time. Let x now be the fraction of the total time that the attacker’s new Gaussian confidence interval for round trip time falls within. If we assume a 3σ ($\approx 99.7\%$) confidence interval, then $x = \frac{6\sigma}{r}$, so $r = \frac{x}{6\sigma}$. The new equation to maximize the information gained by the attacker is:

$$g(x) = x \left(\log_2 \left(\frac{6\sigma}{x} \right) - \frac{1}{2} \log_2 (2\pi e \sigma^2) \right)$$

We are still multiplying the ratio, x , by the difference between two normalized differential entropy formulae, but now the “before”

entropy is normalized to the new notion of a confidence interval ($\pm 3\sigma$) and the “after” entropy is based on the Gaussian distribution for the attacker’s uncertainty about packet delay, $\frac{1}{2} \log_2 (2\pi e \sigma^2)$. Because we do not know *a priori* if the maximum value of $g(x)$ will be independent of σ , we take the partial derivative:

$$\left\{ \frac{\partial}{\partial x} \left(x \left(\log_2 \left(\frac{6\sigma}{x} \right) - \frac{1}{2} \log_2 (2\pi e \sigma^2) \right) \right) = 0, \sigma > 0, 0 < x < 1 \right\}$$

We obtain a maximum at:

$$x = \frac{3\sqrt{\frac{2}{\pi}}}{e^{3/2}} \approx 0.53409633$$

Plugging this value for x back into $g(x)$ gives $g(x) \approx 0.77053915$ bits of information about the identity of the ephemeral port in each round of the attack. This is compared to $f(\frac{1}{e}) \approx 0.53073785$ bits per round if the attacker assumes the worst case, *i.e.*, a uniform distribution.

Note that the maximum in the Gaussian case, *i.e.*, $g(x)$ is independent of σ , meaning that even when we assume a Gaussian distribution for the attacker’s uncertainty about packet delay the effect of adding to that delay is linear in terms of how it affects the rate the attacker will send packets at. Thus the two main takeaway messages of our analysis are:

- Even assuming total equivocation of packet sizes and a worst case scenario for the attacker’s uncertainty about packet delay (uniform distribution), the attacker can gain at least half a bit of information about the ephemeral port in each round of the attack.
- Whether we assume worst case from the attacker’s perspective (uniform) or a more realistic threat model (Gaussian), increasing the variance of the delay only linearly increases the amount of time it takes the attacker to perform one round and has no effect on the information theory and how much information the attacker obtains per round.

In practical terms, the attacker can always learn a 16-bit ephemeral port number within $\left\lceil \frac{16}{0.53073785} \right\rceil = 31$ rounds *based purely on timing information*. This assumes a total equivocation, *i.e.*, 0 bits per round, for the packet size. In practice the VPNs we tested allowed an additional $\log_2(1500) \approx 10.55$ or $\log_2 \frac{1500}{16} \approx 6.55$ bits of information per round, which increases the power of the attack by an order of magnitude. Since every round of the attack costs the attacker roughly a round-trip time and typical attack windows are orders of magnitude larger than a round-trip time, even a very aggressive strategy to add noise to the channel we exploit in this paper will not protect the user.